

مستند الگوریتم تخمین مکان Point Access از داده‌های RSS

استخراج‌شده از کدهای موجود در پروژه

۳۱ تیر ۱۴۰۵

فهرست مطالب

۲	۱ خلاصه
۲	۲ فایل‌های درگیر در الگوریتم
۲	۳ داده ورودی و AP هدف
۲	۴ مختصات محلی متری
۳	۵ مدل ریاضی RSS
۳	۶ بردار پارامترها
۳	۷ تابع باقیمانده و تابع هزینه
۴	۸ مقداردهی اولیه
۴	۹ قیود بهینه‌سازی
۴	۱۰ تبدیل جواب به عرض و طول جغرافیایی
۴	۱۱ معیارهای ارزیابی
۵	۱۲ نتیجه‌ی اجرای فعلی
۵	۱۳ شبیه‌کد الگوریتم
۶	۱۴ نقشه‌ها
۶	۱۵ کد کامل انتخاب AP
۹	۱۶ کد کامل رسم نمونه‌های RSS
۱۱	۱۷ کد کامل تخمین مکان AP
۱۵	۱۸ کد کامل خواندن و رسم نقشه
۱۷	۱۹ نکات مهم استخراج‌شده از کد

۱ خلاصه

در این پروژه مکان یک اکسس پوینت وای فای از روی نمونه های RSS تخمین زده می شود. زنجیره ی کد فعلی از جدول محوری زیر استفاده می کند:

data/pivot_signal_table.csv

اسکرپیت اصلی تخمین مکان، فایل 03_fit_selected_ap_location.py است. دو فایل قبلی برای انتخاب AP و رسم نمونه های خام به کار می روند. فایل faculty_map_json.py برای خواندن نقشه و تبدیل مختصات جغرافیایی به مختصات محلی متری استفاده شده است.

۲ فایل های درگیر در الگوریتم

- 01_select_access_point.py: انتخاب AP هدف با BSSID ثابت، حذف نمونه های نامعتبر و ذخیره ی آمار اولیه.
- 02_plot_selected_ap_from_pivot.py: خواندن AP انتخاب شده و رسم نقشه ی شدت RSS.
- 03_fit_selected_ap_location.py: برازش مدل افت مسیر و تخمین مکان AP.
- faculty_map_json.py: خواندن فایل نقشه و رسم مرز فضاها ی طبقه.

۳ داده ورودی و AP هدف

در کد، AP هدف با شناسه ی زیر انتخاب شده است:

BSSID = b28ba99a8d2a

RSS column = signal_b28ba99a8d2a

هر سطر جدول شامل مختصات نمونه برداری lat, lon و مقدار RSS برای های AP مختلف است. مقدار -100 dBm در این پروژه به عنوان مقدار نامعتبر یا دیده نشدن AP در نظر گرفته شده و نمونه های معتبر با شرط زیر انتخاب می شوند:

$$r_i > -100.$$

برای AP هدف، تعداد نمونه های معتبر برابر 1069 است. آمار RSS استخراج شده از خروجی فعلی پروژه چنین است:

$$\min r_i = -98, \quad \bar{r} = -72.8943, \quad \max r_i = -29.$$

۴ مختصات محلی متری

چون بهینه سازی مکانی باید برحسب متر انجام شود، مختصات جغرافیایی نمونه ها به مختصات محلی تبدیل می شود. اگر (ϕ_i, λ_i) به ترتیب عرض و طول جغرافیایی نمونه ی i باشد و (ϕ_0, λ_0) مبدأ محلی باشد، کد از تقریب زیر استفاده می کند:

$$x_i = (\lambda_i - \lambda_0) 111320 \cos(\phi_0),$$

$$y_i = (\phi_i - \phi_0) 111320.$$

در این روابط زاویه ی ϕ در تابع کسینوس برحسب رادیان محاسبه می شود و x_i, y_i برحسب متر هستند. در مرحله ی برازش، مبدأ محلی میانگین مختصات نمونه های معتبر است:

$$\phi_0 = \frac{1}{N} \sum_{i=1}^N \phi_i, \quad \lambda_0 = \frac{1}{N} \sum_{i=1}^N \lambda_i.$$

۵ مدل ریاضی RSS

مدل به کار رفته در کد، مدل لگاریتمی افت مسیر است:

$$\hat{r}_i = P_m - \gamma \log_{\gamma}(d_i),$$

که در آن:

• $\hat{r}_i$: RSS پیش‌بینی شده در نقطه‌ی نمونه‌ی i ،

• P_m : RSS مدل در فاصله‌ی یک متر،

• n : توان افت مسیر یا path-loss exponent،

• d_i : فاصله‌ی نمونه‌ی i تا مکان مجهول AP.

اگر مکان AP در دستگاه مختصات محلی برابر $\mathbf{a} = (a_x, a_y)$ باشد، فاصله به شکل زیر محاسبه می‌شود:

$$d_i = \max \left(\sqrt{(x_i - a_x)^2 + (y_i - a_y)^2}, \gamma \right).$$

عبارت $\max(\cdot, \gamma)$ در کد برای جلوگیری از فاصله‌ی صفر و ناپایداری $\log_{\gamma}(\cdot)$ است.

۶ بردار پارامترها

پارامترهای مجهول مدل عبارت‌اند از:

$$\boldsymbol{\theta} = [a_x \ a_y \ P_m \ n]^T.$$

هدف الگوریتم پیدا کردن مکان AP یعنی (a_x, a_y) همراه با دو پارامتر رادیویی P_m و n است.

۷ تابع باقیمانده و تابع هزینه

برای هر نمونه، باقیمانده‌ی مدل به صورت اختلاف RSS پیش‌بینی شده و RSS مشاهده شده تعریف می‌شود:

$$e_i(\boldsymbol{\theta}) = \hat{r}_i(\boldsymbol{\theta}) - r_i.$$

در کد، این بردار باقیمانده به تابع `scipy.optimize.least_squares` داده می‌شود:

$$\mathbf{e}(\boldsymbol{\theta}) = [e_1 \ e_2 \ \dots \ e_N]^T.$$

گزینه‌ی `loss="soft_l1"` استفاده شده است. در پیاده‌سازی SciPy، این `loss` به صورت `robust` عمل می‌کند و اثر نمونه‌های پرت را نسبت به کمترین مربعات معمولی کاهش می‌دهد. با مقدار پیش‌فرض `f_scale=1` تابع هزینه‌ی مؤثر را می‌توان به شکل زیر نوشت:

$$\min_{\boldsymbol{\theta}} \frac{1}{2} \sum_{i=1}^N \left(\sqrt{1 + e_i(\boldsymbol{\theta})^2} - 1 \right).$$

۸ مقداردهی اولیه

کد از مختصات ground-truth برای شروع بهینه‌سازی استفاده نمی‌کند. مقدار اولیه‌ی مکان AP از نقطه‌ای گرفته می‌شود که بیشترین RSS را برای AP هدف داشته است:

$$i^* = \arg \max_i r_i,$$

$$a_x^{(\cdot)} = x_{i^*}, \quad a_y^{(\cdot)} = y_{i^*}.$$

دو پارامتر دیگر چنین مقداردهی می‌شوند:

$$P_m^{(\cdot)} = \text{percentile}_{95}(r_i), \quad n^{(\cdot)} = 2.$$

در خروجی فعلی پروژه، مختصات نمونه‌ی شروع برابر است با:

$$\phi_{\text{init}} = 36,3127978, \quad \lambda_{\text{init}} = 59,5263734.$$

۹ قیود بهینه‌سازی

برای جلوگیری از جواب‌های غیرواقعی، کد روی پارامترها کران می‌گذارد:

$$x_{\min} - 30 \leq a_x \leq x_{\max} + 30,$$

$$y_{\min} - 30 \leq a_y \leq y_{\max} + 30,$$

$$-95 \leq P_m \leq -20,$$

$$0,5 \leq n \leq 6.$$

این قیود مکان AP را در محدوده‌ای نزدیک به نقاط نمونه‌برداری نگه می‌دارند و همچنین مقدار توان افت مسیر را در بازه‌ای فیزیکی‌تر محدود می‌کنند.

۱۰ تبدیل جواب به عرض و طول جغرافیایی

پس از تخمین $(\hat{a}_x, \hat{a}_y)$ ، کد آن را دوباره به عرض و طول جغرافیایی تبدیل می‌کند:

$$\hat{\phi} = \phi_0 + \frac{\hat{a}_y}{111320},$$

$$\hat{\lambda} = \lambda_0 + \frac{\hat{a}_x}{111320 \cos(\phi_0)}.$$

۱۱ معیارهای ارزیابی

مختصات واقعی AP در کد فقط برای ارزیابی استفاده شده است، نه برای برازش:

$$\phi_{\text{gt}} = 36,312805555555556, \quad \lambda_{\text{gt}} = 59,52636111111111.$$

خطای مکانی برحسب متر با فاصله‌ی اقلیدسی در مختصات محلی محاسبه می‌شود:

$$E_{\text{loc}} = \sqrt{(\hat{a}_x - a_{x,\text{gt}})^2 + (\hat{a}_y - a_{y,\text{gt}})^2}.$$

همچنین خطاهای RSS چنین محاسبه شده‌اند:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{r}_i - r_i)^2},$$

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\hat{r}_i - r_i|.$$

۱۲ نتیجه‌ی اجرای فعلی

مطابق فایل outputs_rss_ap/03_fit_summary.json نتیجه‌ی اجرای فعلی چنین است:

```
estimated_lat = 36.31281061763301
estimated_lon = 59.5263414104742
ground_truth = (36.312805555555556, 59.52636111111111)
location_error = 1.8548439013658915 m
rss_1m_dbm = -28.360791718620668
path_loss_n = 3.216816335532521
RMSE = 8.53347351228568 dB
MAE = 6.907841020902214 dB
optimizer = success, ftol termination condition is satisfied
```

۱۳ شبکه‌د الگوریتم

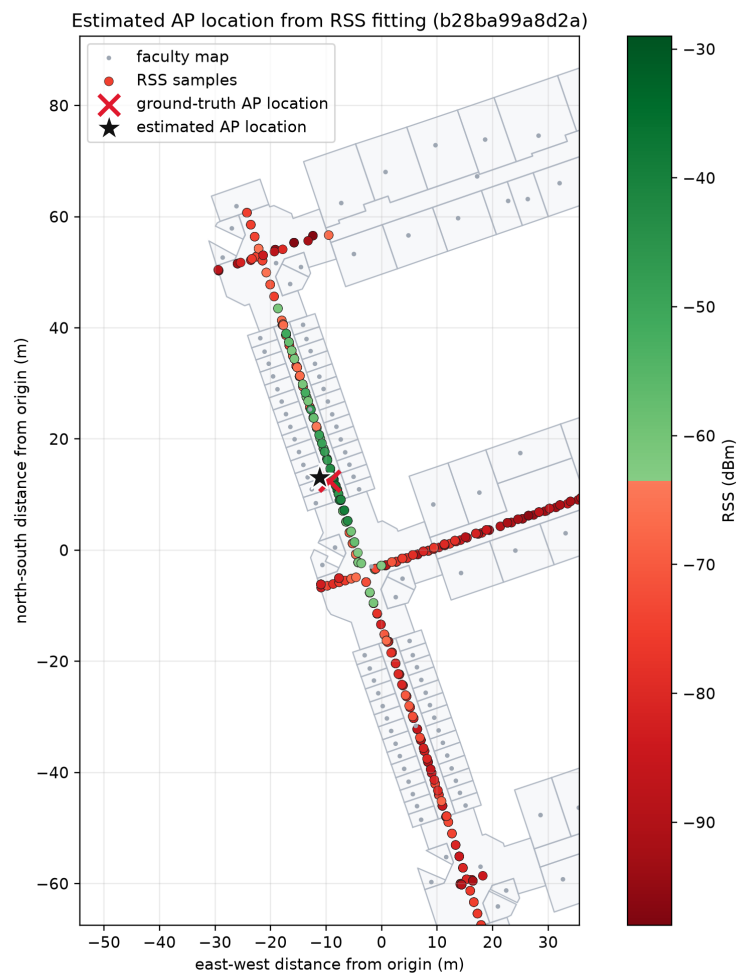
Input:

```
pivot_signal_table.csv
target BSSID = b28ba99a8d2a
```

1. Read pivot RSS table.
2. Select RSS column signal_b28ba99a8d2a.
3. Keep samples whose RSS is greater than -100 dBm.
4. Convert sample lat/lon values to local meter coordinates.
5. Choose the strongest RSS sample as the initial AP position.
6. Initialize:
 ap_x, ap_y = strongest sample position
 rss_1m = 95th percentile of RSS
 n = 2
7. Define log-distance model:
 predicted_rss = rss_1m - 10 n log10(max(distance, 1 m))
8. Minimize robust residuals with scipy least_squares and soft_l1 loss.
9. Convert estimated local AP position back to lat/lon.
10. Compare against ground-truth coordinate only for reporting error.
11. Save summary JSON and draw final map.

۱۴ نقشه‌ها

اگر تصاویر خروجی در پوشه‌ی figures موجود باشند، در این بخش وارد می‌شوند.



شکل ۱: مکان تخمین زده شده‌ی AP روی نقشه، همراه با نمونه‌های RSS و مکان واقعی برای ارزیابی.

۱۵ کد کامل انتخاب AP

```
1 import json
2 import math
3 from pathlib import Path
4
5 import numpy as np
6 import pandas as pd
7
8 from faculty_map_json import read_floor_map
9
10
11 ROOT = Path(__file__).resolve().parent
12 DATA_DIR = ROOT / "data"
13 OUT_DIR = ROOT / "outputs_rss_ap"
```

```

14 PIVOT_DATA = DATA_DIR / "pivot_signal_table.csv"
15
16 # In the experiment dataset, this is the AP under study.
17 SELECTED_AP_BSSID = "b28ba99a8d2a"
18
19 # This coordinate is used only as ground truth for final error
    reporting.
20 GROUND_TRUTH_AP_LAT = 36 + 18 / 60 + 46.1 / 3600
21 GROUND_TRUTH_AP_LON = 59 + 31 / 60 + 34.9 / 3600
22
23
24 def to_local_xy(lat: np.ndarray, lon: np.ndarray, lat0: float, lon0:
    float) -> tuple[np.ndarray, np.ndarray]:
25     """Convert latitude/longitude to local meter coordinates."""
26     meters_per_lat = 111_320.0
27     meters_per_lon = 111_320.0 * math.cos(math.radians(lat0))
28     return (lon - lon0) * meters_per_lon, (lat - lat0) *
        meters_per_lat
29
30
31 def main() -> None:
32     OUT_DIR.mkdir(exist_ok=True)
33
34     # This workflow intentionally uses only the pivot RSS table.
35     df = pd.read_csv(PIVOT_DATA)
36     spaces, _ = read_floor_map(floor_id=4)
37
38     ap_column = f"signal_{SELECTED_AP_BSSID}"
39     if ap_column not in df.columns:
40         raise KeyError(f"{ap_column} was not found in {PIVOT_DATA}")
41
42     valid = df[df[ap_column].astype(float) > -100.0].copy()
43     if valid.empty:
44         raise RuntimeError(f"No valid RSS samples were found for {
            SELECTED_AP_BSSID}.")
45
46     lat0 = float(valid["lat"].mean())
47     lon0 = float(valid["lon"].mean())
48     x, y = to_local_xy(valid["lat"].to_numpy(), valid["lon"].to_numpy(
        ), lat0, lon0)
49     rss = valid[ap_column].to_numpy(dtype=float)
50
51     # This centroid is descriptive only. It is not used as ground
        truth.
52     weights = np.power(10.0, (rss - rss.max()) / 10.0)
53     centroid_x = float(np.average(x, weights=weights))
54     centroid_y = float(np.average(y, weights=weights))
55
56     gt_x, gt_y = to_local_xy(
57         np.array([GROUND_TRUTH_AP_LAT]), np.array([
            GROUND_TRUTH_AP_LON]), lat0, lon0

```

```

58 )
59 centroid_error_m = float(math.hypot(centroid_x - float(gt_x[0]),
60     centroid_y - float(gt_y[0])))
61
62 max_idx = int(np.argmax(rss))
63 max_rss_lat = float(valid.iloc[max_idx]["lat"])
64 max_rss_lon = float(valid.iloc[max_idx]["lon"])
65
66 nearby_spaces = spaces[
67     (spaces["lat"].sub(GROUND_TRUTH_AP_LAT).abs() < 0.00035)
68     & (spaces["lon"].sub(GROUND_TRUTH_AP_LON).abs() < 0.00035)
69 ]["id", "nam", "ref", "lat", "lon"]
70
71 result = {
72     "ground_truth_ap": {
73         "lat": GROUND_TRUTH_AP_LAT,
74         "lon": GROUND_TRUTH_AP_LON,
75         "dms": "36 deg 18'46.1\"N, 59 deg 31'34.9\"E",
76         "usage": "evaluation_only",
77     },
78     "selected_ap": {
79         "ap_column_raw": ap_column,
80         "bssid": SELECTED_AP_BSSID,
81         "num_measurements": int(len(valid)),
82         "max_rss": float(rss.max()),
83         "mean_rss": float(rss.mean()),
84         "min_rss": float(rss.min()),
85         "rss_weighted_centroid_x_m": centroid_x,
86         "rss_weighted_centroid_y_m": centroid_y,
87         "centroid_error_to_ground_truth_m": centroid_error_m,
88         "max_rss_sample_lat": max_rss_lat,
89         "max_rss_sample_lon": max_rss_lon,
90     },
91     "origin": {"lat": lat0, "lon": lon0},
92     "nearby_map_spaces": nearby_spaces.to_dict(orient="records"),
93 }
94
95 (OUT_DIR / "selected_access_point.json").write_text(
96     json.dumps(result, ensure_ascii=False, indent=2), encoding="
97     utf-8"
98 )
99
100 print("Selected AP:", SELECTED_AP_BSSID)
101 print("Valid samples:", len(valid))
102 print("Max RSS sample:", max_rss_lat, max_rss_lon)
103 print("Centroid error to ground truth (m):", round(
104     centroid_error_m, 3))
105 print("Saved:", OUT_DIR / "selected_access_point.json")
106
107 if __name__ == "__main__":

```


106 main()

Listing 1: 01_select_access_point.py

۱۶ کد کامل رسم نمونه‌های RSS

```
1 import json
2 import math
3 from pathlib import Path
4
5 import matplotlib
6
7 matplotlib.use("Agg")
8 import matplotlib.pyplot as plt
9 import numpy as np
10 import pandas as pd
11 from matplotlib.colors import ListedColormap
12
13 from faculty_map_json import draw_floor_map
14
15
16 ROOT = Path(__file__).resolve().parent
17 DATA_DIR = ROOT / "data"
18 OUT_DIR = ROOT / "outputs_rss_ap"
19 PIVOT_DATA = DATA_DIR / "pivot_signal_table.csv"
20 SELECTION_FILE = OUT_DIR / "selected_access_point.json"
21
22
23 def red_green_rss_cmap() -> ListedColormap:
24     """Use only red shades for weak RSS and green shades for strong
25     RSS."""
26     weak_reds = plt.cm.Reds(np.linspace(0.95, 0.45, 128))
27     strong_greens = plt.cm.Greens(np.linspace(0.45, 0.95, 128))
28     return ListedColormap(np.vstack([weak_reds, strong_greens]), name
29     ="rss_red_green")
30
31
32 def to_local_xy(lat: np.ndarray, lon: np.ndarray, lat0: float, lon0:
33 float) -> tuple[np.ndarray, np.ndarray]:
34     """Convert latitude/longitude to local meter coordinates."""
35     meters_per_lat = 111_320.0
36     meters_per_lon = 111_320.0 * math.cos(math.radians(lat0))
37     return (lon - lon0) * meters_per_lon, (lat - lat0) *
38     meters_per_lat
39
40
41 def main() -> None:
42     OUT_DIR.mkdir(exist_ok=True)
```

```

40 # Read the selected AP from step 1 and plot its raw pivot-table
    RSS values.
41 selection = json.loads(SELECTION_FILE.read_text(encoding="utf-8")
    )
42 ground_truth = selection["ground_truth_ap"]
43 ap_column = selection["selected_ap"]["ap_column_raw"]
44 bssid = selection["selected_ap"]["bssid"]
45
46 pivot = pd.read_csv(PIVOT_DATA)
47 if ap_column not in pivot.columns:
48     raise KeyError(f"{ap_column} was not found in {PIVOT_DATA}")
49
50 pivot[ap_column] = pivot[ap_column].astype(float)
51 samples = pivot[pivot[ap_column] > -100.0].copy()
52 samples["rss_dbm"] = samples[ap_column]
53
54 lat0 = float(pivot["lat"].mean())
55 lon0 = float(pivot["lon"].mean())
56 samples["x_m"], samples["y_m"] = to_local_xy(
57     samples["lat"].to_numpy(), samples["lon"].to_numpy(), lat0,
        lon0
58 )
59 target_x, target_y = to_local_xy(
60     np.array([ground_truth["lat"]]), np.array([ground_truth["lon"]
        ])), lat0, lon0
61 )
62
63 fig, ax = plt.subplots(figsize=(9, 9))
64 draw_floor_map(ax, lat0, lon0, floor_id=4)
65
66 points = ax.scatter(
67     samples["x_m"],
68     samples["y_m"],
69     c=samples["rss_dbm"],
70     cmap=red_green_rss_cmap(),
71     s=36,
72     edgecolors="#202020",
73     linewidths=0.35,
74     label="RSS samples",
75 )
76 ax.scatter(
77     target_x,
78     target_y,
79     marker="x",
80     s=180,
81     c="#e0182d",
82     linewidths=2.8,
83     label="ground-truth AP location",
84     zorder=4,
85 )
86

```

```

87     ax.set_xlabel("east-west distance from origin (m)")
88     ax.set_ylabel("north-south distance from origin (m)")
89     ax.grid(True, alpha=0.25)
90     ax.set_aspect("equal", adjustable="box")
91     ax.set_xlim(float(target_x[0]) - 45.0, float(target_x[0]) + 45.0)
92     ax.set_ylim(float(target_y[0]) - 80.0, float(target_y[0]) + 80.0)
93     ax.legend(loc="best")
94     colorbar = fig.colorbar(points, ax=ax)
95     colorbar.set_label("RSS (dBm)")
96     fig.tight_layout()
97     fig.savefig(OUT_DIR / "02_pivot_rss_map.png", dpi=220)
98     plt.close(fig)
99
100     print("AP:", bssid)
101     print("Pivot samples:", len(samples))
102     print("Saved:", OUT_DIR / "02_pivot_rss_map.png")
103
104
105 if __name__ == "__main__":
106     main()

```

Listing 2: 02_plot_selected_ap_from_pivot.py

۱۷ کد کامل تخمین مکان AP

```

1  import json
2  import math
3  from pathlib import Path
4
5  import matplotlib
6
7  matplotlib.use("Agg")
8  import matplotlib.pyplot as plt
9  import numpy as np
10 import pandas as pd
11 from matplotlib.colors import ListedColormap
12 from scipy.optimize import least_squares
13
14 from faculty_map_json import draw_floor_map
15
16
17 ROOT = Path(__file__).resolve().parent
18 DATA_DIR = ROOT / "data"
19 OUT_DIR = ROOT / "outputs_rss_ap"
20 PIVOT_DATA = DATA_DIR / "pivot_signal_table.csv"
21 SELECTION_FILE = OUT_DIR / "selected_access_point.json"
22
23
24 def red_green_rss_cmap() -> ListedColormap:

```

```

25     """Use only red shades for weak RSS and green shades for strong
26     RSS."""
27     weak_reds = plt.cm.Reds(np.linspace(0.95, 0.45, 128))
28     strong_greens = plt.cm.Greens(np.linspace(0.45, 0.95, 128))
29     return ListedColormap(np.vstack([weak_reds, strong_greens]), name
30                               ="rss_red_green")
31
32 def to_local_xy(lat: np.ndarray, lon: np.ndarray, lat0: float, lon0:
33 float) -> tuple[np.ndarray, np.ndarray]:
34     """Convert latitude/longitude to local meter coordinates."""
35     meters_per_lat = 111_320.0
36     meters_per_lon = 111_320.0 * math.cos(math.radians(lat0))
37     return (lon - lon0) * meters_per_lon, (lat - lat0) *
38         meters_per_lat
39
40 def to_lat_lon(x: float, y: float, lat0: float, lon0: float) -> tuple
41 [float, float]:
42     """Convert local meter coordinates back to latitude/longitude."""
43     lat = lat0 + y / 111_320.0
44     lon = lon0 + x / (111_320.0 * math.cos(math.radians(lat0)))
45     return float(lat), float(lon)
46
47 def log_distance_model(params: np.ndarray, x: np.ndarray, y: np.
48 ndarray) -> np.ndarray:
49     """RSS model:  $rss = rss_{1m} - 10 * n * \log_{10}(distance_m)$ ."""
50     ap_x, ap_y, rss_1m, path_loss_n = params
51     distance = np.maximum(np.hypot(x - ap_x, y - ap_y), 1.0)
52     return rss_1m - 10.0 * path_loss_n * np.log10(distance)
53
54 def main() -> None:
55     OUT_DIR.mkdir(exist_ok=True)
56
57     # Fit directly from the pivot table. No train/test or
58     # intermediate CSV is used.
59     selection = json.loads(SESSION_FILE.read_text(encoding="utf-8"))
60
61     ground_truth = selection["ground_truth_ap"]
62     bssid = selection["selected_ap"]["bssid"]
63     ap_column = selection["selected_ap"]["ap_column_raw"]
64
65     pivot = pd.read_csv(PIVOT_DATA)
66     pivot[ap_column] = pivot[ap_column].astype(float)
67     samples = pivot[pivot[ap_column] > -100.0].copy()
68     samples["rss_dbm"] = samples[ap_column]
69
70     lat0 = float(samples["lat"].mean())
71     lon0 = float(samples["lon"].mean())

```

```

68     x, y = to_local_xy(samples["lat"].to_numpy(), samples["lon"].
69         to_numpy(), lat0, lon0)
70     rss = samples["rss_dbm"].to_numpy(dtype=float)
71     gt_x, gt_y = to_local_xy(
72         np.array([ground_truth["lat"]]), np.array([ground_truth["lon"]
73             ])), lat0, lon0
74
75     # The ground-truth AP coordinate is not used by the optimizer.
76     # The initial
77     # AP position is the location where this AP has its strongest RSS
78     sample.
79     max_rss_idx = int(np.argmax(rss))
80     initial = np.array([float(x[max_rss_idx]), float(y[max_rss_idx]),
81         float(np.percentile(rss, 95)), 2.0])
82     lower = [float(x.min() - 30.0), float(y.min() - 30.0), -95.0,
83         0.5]
84     upper = [float(x.max() + 30.0), float(y.max() + 30.0), -20.0,
85         6.0]
86
87     def residual(params: np.ndarray) -> np.ndarray:
88         return log_distance_model(params, x, y) - rss
89
90     result = least_squares(residual, x0=initial, bounds=(lower, upper
91         ), loss="soft_l1")
92     predicted = log_distance_model(result.x, x, y)
93     est_lat, est_lon = to_lat_lon(float(result.x[0]), float(result.x
94         [1]), lat0, lon0)
95     target_error_m = float(
96         math.hypot(float(result.x[0]) - float(gt_x[0]), float(result.
97             x[1]) - float(gt_y[0]))
98     )
99
100     summary = {
101         "bssid": bssid,
102         "estimated_lat": est_lat,
103         "estimated_lon": est_lon,
104         "ground_truth_lat": ground_truth["lat"],
105         "ground_truth_lon": ground_truth["lon"],
106         "distance_from_ground_truth_m": target_error_m,
107         "initial_lat": float(samples.iloc[max_rss_idx]["lat"]),
108         "initial_lon": float(samples.iloc[max_rss_idx]["lon"]),
109         "initial_rule": "maximum_rss_sample",
110         "rss_1m_dbm": float(result.x[2]),
111         "path_loss_n": float(result.x[3]),
112         "rmse_db": float(np.sqrt(np.mean((predicted - rss) ** 2))),
113         "mae_db": float(np.mean(np.abs(predicted - rss))),
114         "success": bool(result.success),
115         "message": str(result.message),
116     }
117     (OUT_DIR / "03_fit_summary.json").write_text(

```

```

109         json.dumps(summary, ensure_ascii=False, indent=2), encoding="
            utf-8"
110     )
111
112     fig, ax = plt.subplots(figsize=(9, 9))
113     draw_floor_map(ax, lat0, lon0, floor_id=4)
114     points = ax.scatter(
115         x,
116         y,
117         c=rss,
118         cmap=red_green_rss_cmap(),
119         s=34,
120         edgecolors="#202020",
121         linewidths=0.3,
122         label="RSS samples",
123     )
124     ax.scatter(
125         gt_x,
126         gt_y,
127         marker="x",
128         s=180,
129         c="#e0182d",
130         linewidths=2.8,
131         label="ground-truth AP location",
132         zorder=4,
133     )
134     ax.scatter(
135         [result.x[0]],
136         [result.x[1]],
137         marker="*",
138         s=330,
139         c="#111111",
140         edgecolors="#ffffff",
141         linewidths=1.1,
142         label="estimated AP location",
143         zorder=5,
144     )
145     ax.set_title(f"Estimated AP location from RSS fitting ({bssid})")
146     ax.set_xlabel("east-west distance from origin (m)")
147     ax.set_ylabel("north-south distance from origin (m)")
148     ax.grid(True, alpha=0.25)
149     ax.set_aspect("equal", adjustable="box")
150     ax.set_xlim(float(gt_x[0]) - 45.0, float(gt_x[0]) + 45.0)
151     ax.set_ylim(float(gt_y[0]) - 80.0, float(gt_y[0]) + 80.0)
152     ax.legend(loc="best")
153     colorbar = fig.colorbar(points, ax=ax)
154     colorbar.set_label("RSS (dBm)")
155     fig.tight_layout()
156     fig.savefig(OUT_DIR / "03_estimated_ap_map.png", dpi=220)
157     plt.close(fig)
158

```

```

159     print("Estimated AP lat/lon:", est_lat, est_lon)
160     print("Distance from ground truth (m):", round(target_error_m, 3)
161           )
162     print("Path-loss n:", round(summary["path_loss_n"], 3))
163     print("RMSE dB:", round(summary["rmse_db"], 3))
164     print("Saved:", OUT_DIR / "03_fit_summary.json")
165
166 if __name__ == "__main__":
167     main()

```

Listing 3: 03_fit_selected_ap_location.py

۱۸ کد کامل خواندن و رسم نقشه

```

1 from __future__ import annotations
2
3 import json
4 import math
5 from pathlib import Path
6
7 import matplotlib.pyplot as plt
8 import numpy as np
9 import pandas as pd
10
11
12 ROOT = Path(__file__).resolve().parent
13 MAP_JSON = ROOT / "data" / "map.json"
14
15
16 def decode_map_text(text: str) -> str:
17     """Decode Persian labels if the database export contains mojibake
18       text."""
19     try:
20         return str(text).encode("latin1").decode("utf-8")
21     except UnicodeError:
22         return str(text)
23
24 def to_local_xy(lat: np.ndarray, lon: np.ndarray, lat0: float, lon0:
25 float) -> tuple[np.ndarray, np.ndarray]:
26     """Convert latitude/longitude to local meter coordinates."""
27     meters_per_lat = 111_320.0
28     meters_per_lon = 111_320.0 * math.cos(math.radians(lat0))
29     x = (lon - lon0) * meters_per_lon
30     y = (lat - lat0) * meters_per_lat
31     return x, y
32
33 def table_from_map_json(table_name: str) -> pd.DataFrame:

```

```

34     """Load one phpMyAdmin-exported table from data/map.json."""
35     data = json.loads(MAP_JSON.read_text(encoding="utf-8"))
36     for item in data:
37         if isinstance(item, dict) and item.get("type") == "table" and
38             item.get("name") == table_name:
39             return pd.DataFrame(item.get("data", []))
40         raise KeyError(f"{table_name} was not found in {MAP_JSON}")
41
42 def read_floor_map(floor_id: int = 4) -> tuple[pd.DataFrame, pd.
43     DataFrame]:
44     """Return map spaces and their polygon coordinates for one floor.
45     """
46     spaces = table_from_map_json("map_space").copy()
47     coords = table_from_map_json("map_space_coordinate").copy()
48
49     for col in ["id", "floor_id", "type_spaces_id"]:
50         spaces[col] = spaces[col].astype(int)
51     for col in ["lat", "lon"]:
52         spaces[col] = spaces[col].astype(float)
53     spaces["nam"] = spaces["nam"].map(decode_map_text)
54     spaces["ref"] = spaces["ref"].map(decode_map_text)
55
56     coords["space_id"] = coords["space_id"].astype(int)
57     coords["rank"] = coords["rank"].astype(int)
58     coords["lat"] = coords["lat"].astype(float)
59     coords["lon"] = coords["lon"].astype(float)
60
61     spaces = spaces[spaces["floor_id"] == floor_id].copy()
62     coords = coords[coords["space_id"].isin(spaces["id"])].copy()
63     return spaces, coords
64
65 def draw_floor_map(
66     ax: plt.Axes,
67     lat0: float,
68     lon0: float,
69     floor_id: int = 4,
70 ) -> pd.DataFrame:
71     """Draw the real map outlines from map_space_coordinate and
72     return spaces."""
73     spaces, coords = read_floor_map(floor_id=floor_id)
74
75     for _, group in coords.sort_values("rank").groupby("space_id"):
76         if len(group) < 2:
77             continue
78         x, y = to_local_xy(group["lat"].to_numpy(), group["lon"].
79             to_numpy(), lat0, lon0)
80         closed_x = np.r_[x, x[0]]
81         closed_y = np.r_[y, y[0]]
82         if len(group) >= 3:

```



```

80         ax.fill(closed_x, closed_y, color="#eef1f5", alpha=0.45,
81                zorder=0)
82     ax.plot(closed_x, closed_y, color="#aeb6c2", linewidth=0.8,
83            alpha=0.95, zorder=1)
84
85     spaces["x_m"], spaces["y_m"] = to_local_xy(
86         spaces["lat"].to_numpy(), spaces["lon"].to_numpy(), lat0,
87         lon0
88     )
89     ax.scatter(
90         spaces["x_m"],
91         spaces["y_m"],
92         s=9,
93         c="#9da6b2",
94         edgecolors="none",
95         label="faculty map",
96         zorder=2,
97     )
98     return spaces

```

Listing 4: faculty_map_json.py

۱۹ نکات مهم استخراج‌شده از کد

- فایل‌های train/test در workflow فعلی استفاده نمی‌شوند؛ ورودی اصلی همان جدول pivot است.
- مقدار ground-truth فقط برای گزارش خطای نهایی است و وارد optimizer نمی‌شود.
- نقطه‌ی شروع optimizer نمونه‌ای است که بیشترین RSS را دارد.
- loss از نوع soft_l1 است، بنابراین برازش نسبت به نویز و نقاط پرت مقاوم‌تر از squares least معمولی است.
- مدل دیوارها، جهت گوشی، چندمسیر بودن سیگنال و تفاوت دستگاه‌ها را صریحاً مدل نمی‌کند؛ همه‌ی این اثرها در باقیمانده‌ی مدل جذب می‌شوند.